

TP Cybersécurité des Systèmes Industriels — Rapport

Analyse, attaque et détection sur une plateforme OT/ICS virtualisée

John — École 2600 (A2/C7 IT-OT)

2026-06-12

1. Résumé exécutif

La plateforme étudiée simule un système de gouverne navale (deux gouvernails *bâbord* et *tribord*) piloté par un automate (PLC) et supervisé par un SCADA au travers d'un Contrôleur-Coordinateur (CC). En fonctionnement nominal, les gouvernails suivent fidèlement l'ordre de barre, qui balaie continûment la plage **0-80**.

L'analyse a montré que le CC expose, sur le réseau host-only, **deux surfaces d'attaque non authentifiées** :

1. une **API web** (/attack, /restore) permettant de manipuler directement les valeurs « métier » de l'automate ;
2. un **serveur Modbus/TCP** (port 502) acceptant des écritures de n'importe quelle source.

La VM *Attacker* exploite la première : un simple GET /attack?plc=gouv&choice=1 &offset=10 répété **fige le gouvernail** (perte de gouverne) tandis que l'ordre de barre continue d'évoluer — une atteinte directe à l'**intégrité** et à la **disponibilité** du procédé. Nous avons reproduit l'attaque, démontré une seconde variante d'**injection de fausses données** (divergence et débordement entier), puis défini une **stratégie de détection orientée métier** : corrélation ordre/gouvernail, détection de valeur figée, et règles réseau sur les écritures Modbus et les appels HTTP /attack.

2. Architecture de la plateforme

2.1 Inventaire des machines

VM	Rôle	Réseau(x)	Adresse(s)
NTP	Référence de temps	host-only	192.168.15.3
PLC	Contrôle de la partie opérative	interne x2	192.168.200.3 / .4
CC	Interface commande ↔ supervision	host-only et interne	192.168.15.1 / 192.168.200.1
SCADA	Visualisation du procédé	host-only	192.168.15.2
Attacker	Poste attaquant	host-only	192.168.15.4

Deux réseaux VirtualBox cohabitent :

- **host-only vboxnet0 — 192.168.15.0/24** : NTP, SCADA, Attacker, et la patte « supervision » du CC.
- **interne intnet — 192.168.200.0/24** : le PLC, et la patte « commande » du CC.

Le **CC est l'unique pont** entre les deux réseaux. Le PLC n'est donc **jamais joignable directement** depuis le segment où se trouve l'attaquant : toute action sur l'automate transite obligatoirement par le CC. C'est un point central de l'analyse.

2.2 Flux de données et carte des registres

Le procédé est porté par des registres Modbus *holding* de l'automate (192.168.200.3). Le CC agit comme **client Modbus** vis-à-vis du PLC, et comme **serveur Modbus + serveur web** vis-à-vis de la supervision.

```

SCADA (.2) --FC6 write reg40 (ordre)--> CC (.1:502)
                                         |
CC (200.1) --FC6 write reg40-----> PLC (200.3)    [ordre de barre]
CC (200.1) --FC3 read reg108/109-----> PLC (200.3)    [gouvernail L/R]
CC (200.1) --FC3 read reg54/56-----> PLC (200.3)    [gouvernail physique L/R]
                                         |
CC web (.1:80) GET / -----+ => JSON metier (teleservice)

```

Carte des registres reconstituée par capture (cf. §4) :

Champ JSON	Registre PLC	Sens	Signification
order_rub	holding 40	écriture	Ordre de barre (consigne)
l_rub / r_rub	holding 108 / 109	lecture	Position gouvernail bâbord / tribord
phy_l_rub / phy_r_rub	holding 54 / 56	lecture	Position physique réelle
fuel	holding 7 (.4)	lecture	Niveau de carburant
(attaque) choice	holding 196	écriture	Mode d'attaque injecté dans le PLC
(attaque) offset	holding 198	écriture	Paramètre d'attaque injecté dans le PLC

Les deux derniers registres ne sont **pas** sollicités en fonctionnement nominal : ce sont les registres qu'écrit le CC dans le PLC lorsqu'on invoque l'API /attack (cf. §5.1). La voie de commande légitime (reg 40) et la voie d'attaque (reg 196/198) sont donc **distinctes**.

L'état métier est consultable en clair via GET <http://192.168.15.1/> :

```
{"l_rub":46,"r_rub":46,"fuel":100,"refuel":0,  
"order_rub":69,"phy_l_rub":46,"energy_speed":0,"phy_r_rub":46}
```

3. Comportement « métier » nominal

En l'absence d'attaque (ordre de démarrage NTP → PLC → CC → SCADA), le SCADA émet en continu un **ordre de barre** qui décrit une **rampe triangulaire** balayant la plage 0-80. Le gouvernail **suit fidèlement** cet ordre : à tout instant, $l_rub \approx r_rub \approx phy_l_rub \approx phy_r_rub \approx order_rub$ (écart ≤ 1).

Extrait de relevé nominal (order mène, le gouvernail suit) :

ts	l_rub	r_rub	order	phy_l	phy_r
17:10:38	26	26	27	26	26
17:10:40	28	28	29	28	28
17:10:42	31	31	31	31	31
17:10:44	32	32	33	32	32

Invariant métier fondamental : *la position du gouvernail suit l'ordre de barre*. C'est cet invariant que l'attaque va briser, et c'est lui qui fonde notre stratégie de détection.

Côté réseau, le trafic nominal est régulier et **prévisible** :

- SCADA (.2) écrit l'ordre (FC6, registre 40) sur le CC, ~1 fois/seconde ; **seule** la .2 réalise cette écriture (85 requêtes .2 → reg40 sur la fenêtre de référence, 0 provenant d'une autre source) ;
- le CC interroge le PLC en lecture (FC3, reg 108/109/54/56) et lui réécrit l'ordre (FC6, reg 40).

4. Méthodologie d'investigation

L'objectif était d'observer **toutes** les communications, y compris le dialogue PLC ↔ CC situé sur le réseau interne, **invisible depuis l'hôte**. La capture hôte classique (tshark -i vboxnet0) ne voit que le segment host-only.

Solution retenue — traçage au niveau de l'hyperviseur. Le CC possédant une patte sur *chaque* réseau, nous avons activé la capture VirtualBox sur ses deux interfaces, ce qui restitue à la fois le segment supervision et le segment commande, sans privilège root sur l'hôte :

```
VBoxManage controlvm CC nictrace1 on # patte interne (PLC)
VBoxManage controlvm CC nictracefile1 cc_intnet.pcap
VBoxManage controlvm CC nictrace2 on # patte host-only (SCADA/Attacker)
VBoxManage controlvm CC nictracefile2 cc_hostonly.pcap
```

Les .pcap ont ensuite été dépouillés avec tshark (dissecteur Modbus) et corrélés au téléservice JSON échantillonné en parallèle. Toute la démarche est rejouable : remise à l'état initial par la procédure documentée (extinction puis redémarrage NTP → PLC → CC → SCADA).

4.1 Pourquoi pas une simple capture sur l'hôte ?

Une capture lancée sur l'interface hôte vboxnet0 ne montre que de l'ARP, de l'ICMP et du SSDP — **jamais** le dialogue Modbus entre VMs. Deux raisons :

1. **Le réseau host-only est un commutateur**. Le trafic *unicast* entre deux VMs (SCADA ↔ CC) est commuté de port à port et n'est **pas** recopié vers le port de l'hôte. L'hôte ne reçoit que le broadcast/multicast et ce qui lui est destiné. Activer le *mode espion* (« promiscuous : Allow All ») sur les adaptateurs des VMs ne change que ce que **les VMs** reçoivent — pas ce que capte l'hôte.
2. **Le réseau interne 192.168.200.0/24 n'a aucune interface côté hôte**. Le dialogue CC ↔ PLC y est donc structurellement hors de portée d'une capture hôte, quel que soit le réglage.

4.2 Capture exhaustive (« god mode ») — toutes les VMs

Pour obtenir une vue *omnisciente* des deux réseaux dans un seul fichier, on **tape directement chaque port virtuel** (et non le commutateur) via le traçage VirtualBox, puis on fusionne les captures en une seule chronologie :

```
for vm in NTP PLC CC SCADA Attacker; do
  VBoxManage controlvm $vm nictrace1 on
  VBoxManage controlvm $vm nictracefile1 /tmp/${vm}_1.pcap
done
# CC et PLC ont une 2e interface (réseau interne) :
VBoxManage controlvm CC nictrace2 on; VBoxManage controlvm CC nictracefile2 /tmp/CC_2.pcap
VBoxManage controlvm PLC nictrace2 on; VBoxManage controlvm PLC nictracefile2 /tmp/PLC_2.pcap

mergcap -w /tmp/godmode.pcap /tmp/*_*.pcap # une seule timeline, 2 réseaux
```

Cette méthode capte **chaque trame émise ou reçue par chaque VM**, sur les deux segments, sans privilège root sur l'hôte et sans modifier la topologie. (Penser à nictrace off en fin d'analyse pour ne pas laisser de traçage résiduel dans la configuration des VMs.)

Note méthodologique. Une première tentative d'injection Modbus brutale (écritures massives sur le registre 40) a **figé l'automate de manière non réversible** par /restore, imposant un redémarrage complet. Sur un système réel où *la disponibilité prime*, ce constat est en soi un résultat : une sonde de test mal calibrée peut provoquer l'incident qu'elle cherche à étudier. Les essais suivants ont été menés de façon incrémentale et restaurés entre chaque variante.

5. Analyse de l'attaque

5.1 Identification

Au lancement de la VM *Attacker* (192.168.15.4), le comportement métier change **immédiatement** : le gouvernail se **fige** (ici à 46, puis 50 selon l'essai) alors que l'ordre de barre, lui, continue de balayer la page.

ts	l_rub	r_rub	order	phy_l	phy_r	
17:12:18	46	46	44	46	46	← gouvernail figé, ordre qui varie
17:12:30	46	46	36	46	46	
17:12:45	46	46	21	46	46	

- **Modification métier** : rupture de l'invariant « le gouvernail suit l'ordre ». La barre ne répond plus → **perte de gouverne**.
- **Machine ciblée** : le **PLC** (la gouverne / plc=gouv), atteint **au travers du CC** — seul pont vers le réseau de commande.

Du clic HTTP à l'écriture Modbus dans le PLC

La capture exhaustive (§4.2) permet de **corrélérer la requête web de l'attaquant avec l'écriture Modbus qui en résulte dans le PLC** — 18 ms plus tard :

```
t=1144.196 s  192.168.15.4 → CC      GET /attack?plc=gouv&choice=1&offset=10
t=1144.214 s  CC → PLC (200.3) FC6   reg198 = 0x000a = 10    <- paramètre "offset"
t=1144.217 s  CC → PLC (200.3) FC6   reg196 = 0x0001 = 1      <- paramètre "choice"
```

Ainsi plc=gouv se traduit, côté commande, par l'écriture des registres **196 (mode/choice)** et **198 (offset)** du PLC. C'est la **commande réellement transmise à l'automate** — sur des registres distincts de la voie d'ordre légitime (reg 40).

Sectionnement de la boucle de commande CC ↔ PLC

Effet de bord majeur, visible au niveau réseau : sous attaque, le dialogue CC ↔ PLC **s'éteint quasi totalement**. La comparaison des captures est sans appel :

État	Trafic Modbus CC ↔ PLC
Nominal	2816 trames / 152 s (~18/s) : 323× lectures FC3 reg 108/109/54/56/6 + 116× écritures FC6 reg 40 (ordre relayé)
Sous attaque	5 trames / 3740 s : la boucle est morte

Le gouvernail se fige donc pour **deux raisons cumulées** : (i) le CC **cesse de relayer l'ordre** vivant vers le PLC (les écritures reg 40 disparaissent), et (ii) la commande de gel (reg 196/198) a été poussée dans le PLC. La valeur affichée est la **dernière position relayée**, gelée. Côté supervision, en revanche, la SCADA continue d'écrire l'ordre sur le CC : seul le brin **CC → PLC** est sectionné.

Important pour l'analyse des captures. Sur une capture prise *sous attaque* (cas du godmode.pcap), on **ne voit donc pas** l'écriture CC → PLC reg 40 : non parce qu'elle n'existe pas, mais parce que l'attaque l'a supprimée. Le relais légitime n'est observable que sur une capture *saine* (clean_intnet.pcap). Il n'existe par ailleurs **jamais** de trame directe SCADA → PLC (0 paquet 15.2 → 200.3) : le CC est un relais applicatif, pas un routeur — il termine la session de supervision et en **ré-origine** une autre vers la commande.

5.2 Vecteur d'attaque

La capture du segment host-only révèle l'action offensive de la VM attaquante — une simple requête web, sans authentification :

192.168.15.4 → 192.168.15.1 GET /attack?plc=gouv&choice=1&offset=10

L'attaquant n'utilise **ni exploit, ni authentification** : il appelle une **API web exposée par le CC**. L'inspection de la spécification OpenAPI (/openapi.json, exposée par FastAPI/uvicorn) confirme trois routes :

Route	Paramètres	Effet
/	—	Téléservice JSON (lecture du procédé)
/attack	plc, choice (int), offset (int)	Manipulation d'une variable du procédé
/restore	plc	Annulation de la manipulation

5.3 Scénarios d'attaque mis en œuvre

Nous avons cartographié l'API par essais successifs (restauration entre chaque) :

- **Scénario A — Gel du gouvernail (déni de gouverne).** choice=1 appelé **en boucle** fige la position du gouvernail à sa valeur courante (30, 46, 50... selon l'instant) tandis que l'ordre continue d'évoluer. C'est l'attaque effectivement portée par la VM *Attacker*. Impact : **perte de disponibilité** de la fonction de gouverne — critique sur un navire.
- **Scénario B — Injection de fausses données (atteinte à l'intégrité).** choice=2 introduit un **biais** entre l'ordre et la valeur remontée (écart mesuré jusqu'à 11). Avec offset=40, la valeur subit un **débordement entier non signé** et remonte à **65510** (≈ −26 sur 16 bits). La supervision affiche alors une position aberrante : un opérateur peut être trompé, ou un automatisme aval déclenché à tort.
- **Scénario C — Injection de commande Modbus.** Indépendamment de l'API web, le serveur Modbus du CC (.1:502) accepte des **écritures non authentifiées**. Une écriture FC6 sur le **registre 40** depuis une source arbitraire force la valeur order_rub :

reg40 ← 65 ⇒ order_rub == 65 (consigne usurpée)

Le protocole Modbus n'offrant **ni authentification ni intégrité**, n'importe quelle machine du segment peut usurper la consigne de barre.

6. Stratégie de détection

Conformément à l'esprit OT (« la donnée a une signification physique, la disponibilité prime »), la détection combine une couche **métier** (la plus robuste) et une couche **réseau**.

6.1 Détection métier (process-aware)

D1 — Rupture de corrélation ordre/gouvernail. C'est le détecteur clé. On surveille l'écart $|\text{order_rub} - \text{phy_l_rub}|$. En nominal il reste ≤ 1 ; sous attaque (gel ou biais) il diverge durablement.

pseudo-règle, évaluée à chaque cycle

```
if abs(order_rub - phy_l_rub) > SEUIL and persiste(> 3 s):  
    alerte("rupture corrélation barre/gouvernail – possible manipulation")
```

D2 — Valeur de procédé figée. Une variance quasi nulle du gouvernail alors que l'ordre varie trahit un actionneur bloqué :

```
if variance(phy_l_rub, fenetre=5s)  $\approx$  0 and variance(order_rub, 5s) > 0:  
    alerte("gouvernail figé – déni de gouverne")
```

D3 — Valeurs hors domaine physique. Le gouvernail évolue dans 0-80. Toute valeur hors plage (ex. 65510) est physiquement impossible : contrôle de domaine systématique sur chaque grandeur.

6.2 Détection réseau (NIDS)

D4 — Écritures Modbus illégitimes. En nominal, **seule la SCADA (.2)** écrit le registre 40. Toute écriture FC6/FC16 provenant d'une autre source, ou visant un registre inattendu, est une anomalie. Règle Suricata indicative :

```
alert modbus any any -> 192.168.15.1 502 (msg:"OT write reg40 source non-SCADA";  
    modbus.function:6; flow:to_server; not src_ip:192.168.15.2; sid:1000041;)
```

D5 — Appels à l'API d'attaque. Tout GET /attack (ou /restore) sur le CC est, en exploitation normale, anormal. Surveillance applicative côté CC + règle HTTP NIDS :

```
alert http any any -> 192.168.15.1 80 (msg:"CC /attack endpoint hit";  
    http.uri; content:"/attack"; sid:1000080;)
```

D6 — Apparition d'un hôte non inventorié. Le démarrage de .4 (l'attaquant) est détectable par supervision ARP/inventaire : toute IP/MAC hors liste blanche sur vboxnet0 lève une alerte.

D7 — Perte du heartbeat de scrutation CC ↔ PLC. La boucle de commande est en nominal extrêmement régulière (~18 trames/s, scrutation FC3 + relais FC6 reg 40). L'attaque la **sectionne** : le trafic CC → PLC s'effondre (2816 trames/152 s → 5 trames/3740 s). Surveiller le **débit de scrutation** sur le brin interne et alarmer dès qu'il chute sous un seuil détecte l'attaque *indépendamment* de la valeur du procédé — y compris si l'attaquant prenait soin de figer une valeur « plausible ». Corollaire : surveiller aussi toute écriture vers les registres **196/198** du PLC, qui n'existent pas en nominal.

D7 : heartbeat de la boucle de commande

```
if taux_scrutation(CC_vers_PLC, fenetre=10s) < SEUIL_MIN:  
    alerte("boucle de commande CC<->PLC sectionnée – relais interrompu")
```

6.3 Synthèse couverture

Détecteur	Scénario A (gel)	Scénario B (biais)	Scénario C (Modbus)
D1 corrélation	✓	✓	✓
D2 valeur figée	✓	—	—
D3 hors-domaine	—	✓	partiel
D4 write Modbus (reg 40 / 196 / 198)	—	—	✓
D5 API /attack	✓	✓	—
D6 inventaire	✓	✓	✓
D7 heartbeat CC↔PLC	✓	✓	—

La redondance métier + réseau garantit qu’aucun scénario n’échappe à au moins deux détecteurs indépendants. **D7** est particulièrement précieux : il détecte le gel à *la source* (interruption du relais), là où un attaquant plus sophistiqué pourrait tromper les détecteurs orientés valeur.

7. Recommandations

1. **Supprimer la surface d'attaque applicative.** Les routes /attack et /restore n'ont aucune raison d'exister en production : à retirer. Plus généralement, le téléservice doit être en **lecture seule** et authentifié.
2. **Segmenter et filtrer.** Le CC étant l'unique pont, il doit appliquer un **filtrage strict** : n'autoriser le port 502 *que* depuis la SCADA, et le port 80 *que* depuis les postes de supervision légitimes. Idéalement, un pare-feu OT (ou une diode de données pour le téléservice) entre supervision et commande.
3. **Sécuriser le Modbus.** Modbus/TCP étant non authentifié, déployer **Modbus/TLS** ou encapsuler dans un tunnel authentifié ; à défaut, verrouiller par ACL et n'autoriser l'écriture que depuis la source légitime, avec contrôle de plage des consignes (la barre $\in [0, 80]$).
4. **Détection orientée métier en production.** Industrialiser D1-D3 dans le CC ou un historian : la corrélation ordre/gouvernail est un capteur d'intégrité peu coûteux et très discriminant.
5. **Sûreté de fonctionnement.** L'automate doit **rejeter une consigne incohérente** (saut brutal, hors plage) plutôt que se figer : un mode dégradé maîtrisé (maintien de cap + alarme) vaut mieux qu'un blocage silencieux de la gouverne.
6. **Inventaire et NAC.** Liste blanche MAC/IP sur les segments OT ; toute machine non inventoriée (l'attaquant) doit être bloquée et alarmée.

8. Conclusion

La plateforme illustre un risque classique des SI industriels : des **fonctions de commande exposées sans authentification** depuis un segment accessible. Sans aucun exploit, un simple appel HTTP suffit à **priver le navire de gouverne**. La défense la plus robuste n'est pas seulement réseau : elle s'appuie sur la **connaissance du procédé** — l'invariant « le gouvernail suit l'ordre » détecte les trois scénarios étudiés, y compris ceux qui n'émettent aucun paquet malveillant identifiable au niveau protocolaire.

Annexes (captures .pcap, scripts d'analyse et d'attaque, relevés) fournies dans le dossier work/.